

Tomato AI Diagnostics

Development Session Log

2026-05-06

Session Overview

Date: 2026-05-06

Repository: nyasha-masendeke/Madomasi

Branch: Madomasi (default branch)

Working directory: C:\Users\PC\Desktop\Madomasi

This session covered a full build-out of the Tomato AI Diagnostics application: backend/frontend integration, live webcam prediction, real-time training curves, two-stage transfer learning, dataset wiring, bug fixes, and project cleanup.

1. Project Structure

```
app.py          - Streamlit entry point
main.py         - CLI entry point (train/evaluate/predict/convert)
pipeline.py     - Core ML: load_datasets, build_model, train_model,
                  train_two_stage, predict_image, evaluate_model,
                  convert_model
config.py       - DISEASE_CLASSES (10 classes), page config, CSS
streamlit_callback.py - Keras callback -> real-time Streamlit charts
components/
  main_ui.py    - Inference tab + Training Dashboard tab
  sidebar.py    - Confidence slider, disease filter, ROI, system stats
src/
  data/dataset.py - 80/10/10 dataset splitter
  utils/recommendations.py - Disease -> treatment text (all 10 classes)
  utils/predictions.py - Simulated inference helper
  utils/training_utils.py - DashboardCallback, get_sys_stats
data/
  raw/raw/tomato/ - 10,584 original images (10 class subfolders)
  splits/         - train (10,532) / val (2,106) / test (2,081)
models/trained/   - Saved Keras checkpoints
models/tflite/    - TFLite exports for edge deployment
outputs/          - Learning curve HTML files (saved after training)
requirements-frontend.txt - streamlit, plotly, pillow, pandas,
                           streamlit-webrtc, av
requirements-backend.txt  - tensorflow, keras, opencv-headless,
                           scikit-learn, psutil, numpy
```

2. Backend / Frontend Integration

Problem: main_ui.py had two run_app() definitions, ~500 lines of dead triple-quoted code, a key mismatch (params['conf'] vs sidebar returning 'confidence'), an undefined 'cb' variable, and a broken progress_bar.progress() call signature.

pipeline.py was an older version using MobileNetV2 and the legacy ImageDataGenerator API with a hardcoded wrong DATA_DIR.

Fixes applied:

- Rewrote main_ui.py: single clean run_app(), fixed all key mismatches, fixed progress bar, wired get_recommendation() from utils.
- Rewrote pipeline.py: MobileNetV3Small + image_dataset_from_directory, sparse_categorical_crossentropy, added evaluate_model() and convert_model().
- Removed st.rerun() from StreamlitTrainCallback.on_epoch_end() -- it was restarting the Streamlit script and killing model.fit() mid-training.
- Filled in CLI stubs in main.py (evaluate, predict, convert).
- Fixed CSS typo: ##46499e -> #46499e.

3. Live Webcam Frame-Level Prediction

Added FramePredictor class to components/main_ui.py:

- Callable for streamlit_webrtc's video_frame_callback
- Loads the Keras model lazily on first frame (avoids blocking UI)
- Runs MobileNetV3 inference every 20 frames (~1.5x/second at 30fps)
- Draws diagnosis overlay (disease name + confidence %) on each frame using OpenCV cv2.putText and cv2.rectangle
- Thread-safe: threading.Lock guards model access (callback runs in a background thread separate from the Streamlit main thread)
- Stored in session_state so model survives Streamlit reruns
- Shows last prediction as metric cards + treatment recommendation below the video stream

Color coding on overlay:

Green = Healthy
Red = Diseased (above confidence threshold)
Orange = Below confidence threshold

Packages added: streamlit-webrtc>=0.47.0, av>=10.0.0

Camera permission note: NotAllowedError: Permission denied is a browser security restriction, not a code bug. Requires user to click Allow on the browser permission popup. HTTPS required for non-localhost URLs.

4. Real-Time Learning Curves

Updated StreamlitTrainCallback to accept a chart_placeholder parameter.

On every epoch end:

1. Progress bar updates: 'Epoch N / Total'
2. Four metric cards: Train Acc | Train Loss | Val Acc | Val Loss
3. Plotly 2-subplot chart redraws in place:
 - Left: Accuracy (train=green, val=orange-dashed)
 - Right: Loss (train=red, val=blue-dashed)

After training completes, _save_learning_curves() writes an interactive standalone HTML file to outputs/learning_curves_<timestamp>.html using fig.write_html().

Fixed default Dataset Directory in UI: 'data/train' -> 'data/splits/train'.

5. Dataset Wiring

Dataset location: data/raw/raw/tomato/ (double 'raw' in path)
10 class folders, ~1,100 images each (total: 10,584 images)

Classes (sorted alphabetically = Keras index order):

- 0 Tomato__Bacterial_spot
- 1 Tomato__Early_blight
- 2 Tomato__healthy
- 3 Tomato__Late_blight
- 4 Tomato__Leaf_Mold
- 5 Tomato__Septoria_leaf_spot
- 6 Tomato__Spider_mites Two-spotted_spider_mite
- 7 Tomato__Target_Spot
- 8 Tomato__Tomato_mosaic_virus
- 9 Tomato__Tomato_Yellow_Leaf_Curl_Virus

Split (80/10/10 via src/data/dataset.py):

Train: 10,532 | Val: 2,106 | Test: 2,081

Pipeline verification: load_datasets() + build_model() tested end-to-end.
Model output shape confirmed: (None, 10)

Bug fixed: class_names attribute lost after .prefetch() call.
Fix: cache class_names before prefetch and re-attach after.

6. Two-Stage Transfer Learning

Added train_two_stage() to pipeline.py:

Stage 1 - Feature Extraction:

Base model: MobileNetV3Small FROZEN (ImageNet weights preserved)
Trains: classification head only (GlobalAvgPool -> Dropout -> Dense(10))
Learning rate: 0.001
Saves best checkpoint by val_accuracy

Stage 2 - Fine-Tuning:

Reloads best Stage 1 checkpoint
Base model: UNFROZEN (entire network trains end-to-end)
Learning rate: 0.0001 (10x lower to avoid destroying ImageNet features)
Saves best checkpoint by val_accuracy (overwrites if better)

Both stages use:

ModelCheckpoint (save_best_only=True, monitor=val_accuracy)
EarlyStopping (patience=3, monitor=val_loss, restore_best_weights=True)

UI: Training Dashboard now has a 'Two-Stage Training' toggle (default ON)
with separate epoch/LR controls for each stage. Falls back to single-stage
train_model() when toggled OFF.

7. Bug Fixes

StreamlitAPIException - multiselect default mismatch:

sidebar.py had hardcoded default=['Early Blight', 'Late Blight', ...]
after DISEASE_CLASSES was updated to Tomato___* folder names.
Fix: changed default=DISEASE_CLASSES (all classes selected by default).

Stale 'Healthy' string comparisons:

main_ui.py and sidebar.py used == 'Healthy' in multiple places.
Fix: changed to 'healthy' in disease.lower() to match 'Tomato___healthy'.

recommendations.py stale keys:

Dict keys used old display names ('Early Blight', etc.).
Fix: rewrote all keys to match actual folder names, added all 10 classes.

dataset.py UnicodeEncodeError:

print() with emoji caused cp1252 encoding error on Windows.
Fix: removed emoji from print statement.

dataset.py wrong raw path:

Hardcoded 'data/raw/tomato' -- actual path is 'data/raw/raw/tomato'.
Fix: updated path in split_dataset() call.

8. Project Cleanup

Deleted 4 duplicate 'Copy' files (217 lines of dead code removed):

- config - Copy.py
- main - Copy.py
- pipeline - Copy.py
- streamlit_callback - Copy.py

.gitignore improvements:

- models/**/*.*.keras, *.tflite, *.h5 (catch models in any subdirectory)
- outputs/*.html, outputs/*.png (prevent learning curve files from committing)

README.md added with:

- Feature overview
- Full disease class table
- Project structure tree
- Quick start guide
- CLI training commands
- Two-stage training explanation
- Edge deployment (Raspberry Pi 5) instructions
- Webcam permission notes
- Requirements breakdown

9. Commit History

c0aa4b3 Remove Copy files and tighten .gitignore
f241837 Add README.md
7612eb4 Add camera permission guidance for webcam tab
d199e8d Fix stale class name references after DISEASE_CLASSES update
b5c2825 Add two-stage feature extraction and fine-tuning pipeline

79c782d Add real-time learning curves during training + save to outputs/
719826e Wire dataset and fix pipeline for 10-class tomato dataset
71ce3d0 Add live webcam frame-level disease prediction
caf9328 Resolve merge conflicts: keep fixed versions from claude branch
e1efba1 Merge claude/naughty-benz-c78141: fix backend/frontend integration
8befc6e Fix backend/frontend integration: rewrite main_ui.py and pipeline.py

10. Outstanding Professional Suggestions

High priority:

- Save class_names inside model metadata at training time to eliminate manual DISEASE_CLASSES sync requirement
- Add data augmentation to load_datasets() (flips, rotation, brightness)
- Add display name mapping for class folder names in the UI

Medium priority:

- Add per-class precision/recall/F1 to evaluate_model()
- Add confidence calibration (temperature scaling) post-training
- Add model status indicator in UI (green/red: model file exists or not)
- Webcam scans should increment session statistics footer

Lower priority:

- Add FastAPI layer for serving predictions to other clients
- Add model versioning registry (models/registry.json)
- Centralise all path constants in config.py or .env
- Write real smoke tests in tests/test_smoke.py